

Práctica 06. Mantenimiento de Llaves Foráneas

Gallardo Valdez Brayan Alexis Granados Rovira José Pablo
Lozano León Diego Márquez Corona Danna Lizette
Paredes González Emiliano Sebastián Tapia Anrubio Irving Axel

20 Abril 2026

Investigar políticas

Preguntas de laboratorio

1. ¿Qué es una política de mantenimiento de llaves foráneas?
Es la acción referencial que define qué debe ocurrir cuando una fila de la tabla referenciada se elimina o cuando su llave se actualiza, y existen filas relacionadas en la tabla que referencia esa llave foránea. Las acciones se declaran con **ON DELETE** y **ON UPDATE**, y PostgreSQL contempla las opciones **NO ACTION**, **RESTRICT**, **CASCADE**, **SET NULL** y **SET DEFAULT**.
2. Para cada política que investigaron, ¿cómo se indica en SQL?
 - NO ACTION
 - ON DELETE SET DEFAULT
 - ON UPDATE SET DEFAULT
 - RESTRICT
 - ON DELETE SET DEFAULT
 - ON UPDATE SET DEFAULT
 - CASCADE
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - SET NULL
 - ON DELETE SET NULL
 - ON UPDATE SET NULL

- SET DEFAULT
 - ON DELETE SET DEFAULT
 - ON UPDATE SET DEFAULT

3. Para cada política que investigaron, ¿cuál es su objeto y su funcionamiento?

- NO ACTION
Lanza un error si la acción rompe la integridad referencial al final de la transacción.
- RESTRICT
No permite eliminar o actualizar una fila si existen registros que dependen de ella.
- CASCADE
Si se elimina o actualiza una fila en la tabla padre, automáticamente se eliminan o actualizan las filas relacionadas en la tabla hija.
- SET NULL
Cuando se elimina o actualiza la fila padre, las llaves foráneas en la tabla hija se establecen en NULL
- SET DEFAULT
La llave foránea toma un valor por defecto definido previamente.

4. Para cada política que investigaron, ¿cuáles son sus ventajas y desventajas?

- NO ACTION
Ventajas: Esta política es mucho más flexible y compleja que las demás, permitiendo hacer cambios a varias tablas dentro de una misma operación sin romperse a la mitad de ella. Cuando se quieren hacer scripts masivos que no se interrumpan a la mitad, esta es la ideal.
Desventajas: Se necesita más conocimiento de causa para poder realizar esta política pues puede ser complicado encontrar el error ya que no se interrumpe justo donde pasa el error. Como en todo, cuando se gana flexibilidad, se pierde estructura y se aumenta la complejidad.
- RESTRICT
Ventajas: La ventaja principal es que protege bien los datos y es el que más se preocupa en la integridad de los datos. Es ideal para esquemas donde hay un historial importante como lo puede ser una farmacia que es importante el historial médico, o en una escuela.
Desventajas: La desventaja principal es lo mismo que su ventaja principal, la rigidez que impone en las llaves foráneas hace tedioso el tener que borrar una que necesariamente debe ser eliminado pues se deben de borrar primero a los hijos. Cuando tenemos detalles simples como correos o teléfonos no es lo recomendable.

- **CASCADE**

Ventajas: Lo mejor de esta política de mantenimiento es el hecho de la automatización de la limpieza de datos pues si algo ya no te sirve, sus hijos menos y siguiendo esta lógica, esta política de mantenimiento es ideal. Además con esta política es imposible que exista un hijo sin padre, lo cual evita situaciones indeseables en integridad de datos.

Desventajas: Los efectos secundarios de un error son catastróficos con esta política pues si borras algún dato importante, no pierdes solo esa información sino también cualquier información que derivaba de ella.

- **SET NULL**

Ventajas: A diferencia del anterior, si borras al padre por error, aquí no pierdes la información del hijo aumentando un poco la flexibilidad que puede tener la base de datos.

Desventajas: Esta es de las que más se meten con la integridad de los datos pues puedes perder casi toda la lógica del sistema borrando cosas que no se deben de borrar, perdiendo relaciones entre datos y generando inconsistencias terribles.

- **SET DEFAULT**

Ventajas: Con este se mantiene la estructura de la base de datos lo que ayuda a que no pierdas relaciones importantes. **Desventajas:** Se agregan valores artificiales que no necesariamente tienen algún sentido en tu base de datos lo cual puede hacer difuso el paso de la extracción de información a partir de ella.

5. Con base en lo anterior, ¿cuál política utilizarán para su esquema, y por qué motivo? Vamos a usar no solamente una pues creemos que al usar solamente una, no estamos considerando los casos específicos de cada una de las llaves foráneas. Para atributos multivaluados vamos a usar **CASCADE** pues no tienen sentido si se eliminan a sus padres, como por ejemplo, no tiene sentido guardar los teléfonos de un cliente si borramos al cliente. Asimismo, lo haremos **CASCADE** con aquellas relaciones como **PEDIR** y **GENERAR** que son un puente entre dos cosas y si se borra el padre, no tiene sentido que existan. Luego entonces, para las demás relaciones importantes donde sí nos importa guardar el historial del cliente como **Ticket** y **Cliente**, o **Consulta** y **Cliente** utilizaremos **RESTRICT**, note que esta es la que más usaremos porque como farmacia nos interesa la integridad de los datos y el historial de prácticamente todo: **Cientes**, **Sucursales**, **Personal**, **Medicamentos**, etc.

Referencias

- [1] PostgreSQL Global Development Group. (s. f.). *Documentation: 18: 5.5. Constraints*. PostgreSQL Documentation. Sitio web: <https://www.postgresql.org/docs/current/ddl-constraints.html>
- [2] PostgreSQL Global Development Group. (s. f.). *Documentation: 18: CREATE TABLE*. PostgreSQL Documentation. Sitio web: <https://www.postgresql.org/docs/current/sql-createtable.html>
- [3] PostgreSQL Global Development Group. (s. f.). *Documentation: 18: ALTER TABLE*. PostgreSQL Documentation. Sitio web: <https://www.postgresql.org/docs/current/sql-altertable.html>
- [4] PostgreSQL Global Development Group. (s. f.). *Documentation: 18: 3.3. Foreign Keys*. PostgreSQL Documentation. Sitio web: <https://www.postgresql.org/docs/current/tutorial-fk.html>